UniCaronas

Sistema de Caronas Universitárias

Resumo da Reunião de Planejamento — Sprint 3

Engenharia de Software — 3ESCN

Período da Sprint: 03/04/2026 a 10/04/2026 (1 semana)

Função	Nome
Product Owner	Rafael Machado
Scrum Master	Ariane da Silva Archanjo
Dev Team	Pedro Kafka, Matheus Sizanoski, Rafael Machado, Ariane da Silva Archanjo

---

1. Como a cerimônia aconteceu

A reunião de Sprint Planning foi realizada em 09/04/2026 de forma presencial. A Scrum Master conduziu o planejamento, o Product Owner apresentou as funcionalidades priorizadas para esta sprint, e a equipe estimou, discutiu e se comprometeu com as entregas. Dado que a Sprint tem duração de apenas uma semana, o foco foi mantido em escopo reduzido e bem definido.

Campo	Detalhe
Data	01/04/2026 (quarta-feira)
Formato	Presencial
Duração	Aproximadamente 1h30
Quem conduziu	Ariane da Silva Archanjo (Scrum Master)
Quem definiu as tarefas	Rafael Machado (Product Owner)
Início da Sprint	03/04/2026
Fim da Sprint	10/04/2026 — próxima aula (algo funcional já entregue)

---

2. O que será entregue ao final da Sprint

Ao final desta Sprint, usuários com solicitação aceita devem conseguir trocar mensagens em tempo real com o motorista dentro da plataforma, sem depender de aplicativos externos. O motorista também poderá aceitar ou recusar solicitações diretamente pela interface.

A entrega está completa quando:

- Chat funcional entre motorista e passageiro — mensagens enviadas e recebidas em tempo real (polling)
- Motorista consegue aceitar ou recusar solicitações de vaga pela tela de detalhes da carona

- Histórico de mensagens carregado ao abrir o chat
- Indicação visual de mensagens não lidas
- Endpoint de pagamento integrado à tela de pagamento (fluxo simulado)
- Tela de pagamento funcional com resumo de valor, taxa e repasse
- Código salvo no GitHub e fluxo completo testado no navegador

---

3. Quais histórias de usuário serão trabalhadas

O Product Owner apresentou três histórias com base no backlog priorizado. Dado o prazo reduzido de uma semana, o escopo foi ajustado para garantir entrega de valor sem comprometer a qualidade.

ID	Funcionalidade	Pontos estimados	Prioridade	Sprint
US08	Chat entre usuários (motorista e passageiro)	5 pontos	Alta	3
US09	Realizar pagamento	5 pontos	Alta	3
US08b	Aceitar/Recusar solicitação pela interface	3 pontos	Alta	3
—	Total	13 pontos	—	—

---

4. Por quem será feito

O trabalho é da equipe como um todo. Todos participaram do planejamento, concordaram com as tarefas e se comprometeram com a entrega.

Membro	Responsabilidade principal
Rafael Machado	Product Owner: refinamento de requisitos, desenvolvimento do backend de pagamentos e endpoint de resposta a solicitações
Ariane da Silva Archanjo	Scrum Master: facilitação do processo, desenvolvimento da tela de chat e integração frontend
Pedro Kafka	Desenvolvimento: backend do chat (mensagensController), rotas e lógica de acesso
Matheus Sizanoski	Desenvolvimento: tela de pagamento, integração com API e testes manuais do fluxo completo

---

5. Como cada funcionalidade será feita

US08 — Chat entre usuários (5 pontos)

Como motorista ou passageiro, quero me comunicar via chat com o outro usuário após a solicitação, para combinar detalhes como ponto de encontro e horário exato.

Como será feito:

- A tabela mensagens_chat já existe no banco — nenhuma migração necessária
- Usar os endpoints já implementados: POST /api/mensagens e GET /api/mensagens/:solicitacao_id
- Frontend faz polling a cada 5 segundos para verificar novas mensagens (setInterval)
- Chat exibido na tela carona.html após solicitação aceita, dentro da coluna lateral
- Mensagens do usuário logado alinhadas à direita (bolha roxa), do outro à esquerda (bolha escura)
- Histórico completo carregado ao abrir — mensagens marcadas como lidas automaticamente via backend
- Enter no campo de texto envia a mensagem — botão Enviar disponível para mouse
- Indicação visual do nome do remetente para mensagens recebidas

Para considerar pronto:

Critério de Aceitação	Status
Chat disponível após solicitação aceita pelo motorista	■ Planejado
Histórico de mensagens carregado ao abrir o chat	■ Planejado
Mensagens novas aparecem sem recarregar a página (polling 5s)	■ Planejado
Indicação visual de mensagens não lidas (badge na navbar)	■ Planejado
Apenas motorista e passageiro da carona acessam o chat	■ Planejado

US08b — Aceitar/Recusar solicitação pela interface (3 pontos)

Como motorista, quero visualizar solicitações pendentes e aceitar ou recusar cada uma diretamente na tela, sem precisar usar ferramentas externas.

Como será feito:

- O endpoint PATCH /api/caronas/solicitacoes/:id já está implementado no backend
- Adicionar seção 'Solicitações pendentes' na tela carona.html — visível apenas para o motorista
- Buscar solicitações pendentes com GET /api/caronas/:id que retorna a carona com dados do motorista
- Criar endpoint GET /api/caronas/:id/solicitacoes para listar solicitações com nome e avaliação do passageiro
- Botões Aceitar e Recusar disparam PATCH /api/caronas/solicitacoes/:id com status correspondente
- Após ação, lista de solicitações é atualizada na tela sem recarregar
- Trigger no banco atualiza vagas_disponiveis automaticamente ao aceitar

Para considerar pronto:

Critério de Aceitação	Status
Motorista vê lista de solicitações pendentes na tela da carona	■ Planejado
Botões Aceitar e Recusar funcionam e atualizam o status	■ Planejado
Vaga é descontada ao aceitar; devolvida ao recusar	■ Planejado
Passageiros não veem a seção de gerenciamento de solicitações	■ Planejado

US09 — Realizar pagamento (5 pontos)

Como passageiro com vaga confirmada, quero realizar o pagamento dentro da plataforma, para que não precise lidar com dinheiro em espécie e a transação seja segura.

Como será feito:

- O controller pagamentosController.js já está implementado com lógica de taxa (10%) e repasse
- Criar tela pagamento.html com resumo da carona, detalhamento de valores e seleção de método
- Exibir: valor total, taxa da plataforma (10%) e repasse ao motorista
- Métodos disponíveis: cartão de crédito, PIX e dinheiro (simulados — sem integração real)
- Botão Confirmar pagamento chama POST /api/pagamentos e exibe confirmação
- Após pagamento, redirecionar para carona.html com status atualizado
- Verificar se já existe pagamento (status 409) e bloquear duplo pagamento
- Link para tela de pagamento exibido no chat após solicitação aceita

Para considerar pronto:

Critério de Aceitação	Status
Tela exibe detalhamento de valores (total, taxa, repasse)	■ Planejado
Passageiro seleciona método e confirma pagamento	■ Planejado
Sistema retém 10% como taxa da plataforma	■ Planejado
Duplo pagamento bloqueado com mensagem de erro clara	■ Planejado
Confirmação visual após pagamento bem-sucedido	■ Planejado

---

6. Tecnologias

Tecnologia	Para que será usada
HTML, CSS e JavaScript	Telas de chat (carona.html atualizado) e pagamento (pagamento.html nova)
Node.js + Express	Endpoint de listagem de solicitações e integração dos endpoints existentes de chat e pagamento
PostgreSQL	Tabelas mensagens_chat e pagamentos — já existentes, sem migração
JWT (token de acesso)	Autenticação nas rotas de chat e pagamento
setInterval / polling	Atualização automática de mensagens a cada 5 segundos
GitHub	Versionamento — entrega na branch principal

---

7. Quando será testado

Os testes são manuais: a equipe usa o próprio navegador para verificar se cada funcionalidade está funcionando como esperado. Dado o prazo de uma semana, os testes são integrados ao desenvolvimento diário.

Data	O que é testado	Como é testado
08/04	US08b — Aceitar/Recusar solicitação	Motorista visualiza solicitação pendente e clica em Aceitar; verificar se vaga diminui no banco
08/04	US08 — Chat	Dois usuários trocam mensagens; verificar histórico, polling e acesso restrito
08/04	US09 — Pagamento	Passageiro completa fluxo de pagamento; verificar taxa, confirmação e bloqueio de duplo pagamento
08/04	Fluxo completo	Criar carona → buscar → solicitar → aceitar → chat → pagar. Product Owner valida
08/04	Correções	Corrigir qualquer problema encontrado nos dias anteriores

---

8. Quando vai para produção

Nesta Sprint, 'ir para produção' significa ter o código finalizado no GitHub e uma versão funcionando para apresentar em aula.

Data	O que acontece
09/04	Código finalizado e salvo na branch principal do GitHub — versão estável e sem erros conhecidos
09/04	Apresentação do sistema funcionando no navegador, com o fluxo solicitar → aceitar → chat → pagar